Laboratorio 01

Tema: OmegaUp

Alumno	Carrera Profesional	Asignatura
Diego Humberto Ccoyaccoya dccoyaccoyh@ulasalle.edu.pe	Carrera Profesional de Ingeniería de Software	Lenguaje de Programación I

Laboratorio	Tema	Duración
01	OmegaUp	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - I	8 Abril 2026	9 Abril 2026

Índice

1. Especificaciones del Laboratorio **2**
1.1. Objetivos del laboratorio . . . 2
1.2. Equipos, materiales y temas . . . 2

2. Marco teórico **2**
2.1. OmegaUp . . . 2
2.2. Contratación Dinámica . . . 2

3. Guía de laboratorio **4**
3.1. Directorio de trabajo personal . . . 4
3.2. Repositorio local . . . 4
3.3. Directorio del laboratorio . . . 4
3.4. Archivo .gitignore . . . 5
3.4.1. Envío a omegaUp . . . 5

4. Tarea **6**
4.1. Descripción . . . 6
4.2. Pregunta . . . 6
4.3. Entregables . . . 7

5. Rúbricas **7**
5.1. Contenido del Informe . . . 7

6. Referencias **9**

1. Especificaciones del Laboratorio

1.1. Objetivos del laboratorio

- Analizar un problema de programación.
- Diseñar pseudocódigos, diagramas de flujo, etc. que permitan describir la solución que queremos proponer.
- Implementar todas las soluciones factibles en un lenguaje de programación.
- Envíar sus implementaciones a la plataforma OmegaUp.

1.2. Equipos, materiales y temas

- Sistema Operativo `Windows`: Windows 10
- Editor de texto plano `C++` 5.11
- Cuenta en `GitHub` asociada al correo institucional.

2. Marco teórico

2.1. OmegaUp

- OmegaUp es una organización sin fines de lucro dedicada a brindar a los estudiantes desatendidos de América Latina las habilidades y el conocimiento que necesitan para tener éxito en el mundo actual impulsado por la tecnología.
- omegaUp ofrece contenido educativo gratuito de Ciencias de la Computación de alta calidad a más de 50.000 estudiantes en América Latina a través de nuestra plataforma en línea https://omegaup.com/.
- Participa en concursos de programación y pon a prueba tus conocimientos.
- Resuelve problemas y mejora tus habilidades.
- Organiza tu concurso con problemas que están disponibles en la plataforma o crea tus propios problemas.
- Sigue el progreso de tus estudiantes a través de tareas y exámenes que se evalúan automáticamente.

2.2. Contratación Dinámica

- ID: 22663
- Título: Contratación Dinámica
- Nivel Básico: Introducción en la programación.
- Calidad: Bueno.
- Dificultad: Medio.
- Ratio: 25.63

- Temas: Cálculos aitméticos, Ciclos, Implementación, Recursión.

Tabla 1: 22663 Contratación Dinámica

Puntos	12.26	**Límite de Memoria**	32 MiB
Límite de tiempo (caso)	1s	**Límite de tiempo (total)**	1m0s
Tamaño límite de entrada (bytes)	48828.125 KiB		

- Daniel, el gerente de la biblioteca, necesita un método para decidir cuántos empleados de reserva necesita cada día. Para optimizar recursos, basa su decisión en el número de solicitudes de los días más recientes.
- Su sistema funciona de la siguiente manera: cada día, analiza un período que abarca los últimos K días de actividad (incluyendo el día actual). Dentro de ese período, identifica la jornada con el mayor número de solicitudes.
- `Por ejemplo`: Considere el entero poditivo 24. Sumar 2 y 4 resulta en 6. Ya que 6 es un solo digito, 6 es la raiz digital de 24. Ahora considere el entero positivo 39. Sumar 3 y 9 resulta en 12. Ya que 12 no es un solo digito, el proceso debe ser repetido. Sumando 1 y 2 resulta en 3, de un solo digito y por tanto la raiz digital de 39.
- `Entrada`
- La primera línea contiene dos enteros N y K, el total de días registrados y la duración del período de análisis, respectivamente.
- La segunda línea contiene N enteros, que representan el número de solicitudes para el día i .

Tabla 2: 22663.Contratación Dinámica

Entrada	**Salida**
4	6
39	3
0	

3. Guía de laboratorio

3.1. Directorio de trabajo personal

- Cree su directorio de trabajo personal. (Debe utilizar su usuario institucional.)
- Luego, diríjase a este directorio.

Listing 1: Creando directorio personal

```
$ mkdir -p $HOME/r.escobedo/
```

Listing 2: Dirijíendonos al directorio personal

```
$ cd $HOME/r.escobedo/
```

3.2. Repositorio local

- Clone su repositorio remoto.
- Luego, diríjase a este directorio.

Listing 3: Clonando repositoio remoto

```
$ git clone https://github.com/DiegoCcoyaccoya/lp1.git
```

Listing 4: Dirijíendonos al directorio del repositorio local

```
```

3.3. Directorio del laboratorio

- Cree un directorio para el laboratorio correspondiente, dentro del directorio para el repositorio local. (Utilizar el estándar)
- Luego, diríjase a este directorio.

Listing 5: Creando directorio para este laboratorio

```
$ mkdir -p $HOME/r.escobedo/lp1/lab01/exercises/
```

Listing 6: Dirijíendonos al directorio del laboratorio

```
$ cd $HOME/r.escobedo/lp1/lab01
```

3.4. Archivo .gitignore

- Siempre evalue utilizar el archivo `.gitignore` para no hacer seguimiento a archivos innecesarios, esto es muy importante sobre todo para ignorarlos en el repositorio GitHub.
- Pueden haber varios de estos archivos y estar ubicados estratégicamente en algún directorio; por ejemplo sólo para el laboratorio o para todo el repositorio local.

Listing 7: Creando .gitignore específico para el laboratorio

```
$ vim $HOME/r.escobedo/lp1/lab01/.gitignore
```

3.4.1. Envío a omegaUp

- Para envíar la implementación de sus propuestas, primero seleccione el lenguaje de programación utilizado.
  - C++20 (g++ 10.3)
  - C# (10, dotnet 6.0)
  - Go (1.18.beta2)
  - Haskell (ghc 8.8)
  - **Java (openjdk 16.0)**
  - JavaScript (Node.js 16)
  - Kotlin (1.6.10)
  - Lua (5.3)
  - Python (3.9)
  - Ruby (2.7)
  - Rust (1.56.1)
- Luego, copie el código en la pestaña `code: Main.java`. Recuerde que en `Java el nombre de la clase que tiene el método main() debe ser el mismo que el nombre del archivo *.java.`
- `Ejecute el código haciendo clic en el botón Run.`
- `Si no hay errores, envíe la solución, haciendo clic en el botón Submit.`
- `En la parte inferior ud. podra visualizar el estado de sus envíos.`

Fecha y hora	GUID	Estatus	Porcentaje	Lenguaje	Memoria	Tiempo	Acciones
2023-08-29 12:17	55c7d730	AC	100.00%	java	0.00 MB	0.07 s	
2023-08-29 12:16	8a811457	AC	100.00%	java	0.00 MB	0.07 s	
2023-08-29 12:14	6b027854	AC	100.00%	java	0.00 MB	0.09 s	
Nuevo envío							

Figura 1: Envíos en omegaUp

4. Tarea

4.1. Descripción

- Elabore un informe individual, para otro problema, reutilizando la presente guía de laboratorio pero con el lenguaje de programación C++.
- Siempre piense en las posibilidades de realizar otras implemetaciones particulares.
- Envíe satisfactoriamente todas sus implementaciones a la plataforma omegaUp.
- Utilice todas las recomendaciones dadas por el docente:
  - Antecedentes: Describir antecedentes previos que sean necesarios para desarrollar el laboratorio. (Las entregadas por el docente y/o las que se buscaron personalmente).
  - Commits: Elaborar la lista de envíos que permitiran culminar el laboratorio. (Previo a la implementación)
  - Source: Explicar porciones de código fuente importantes, trascendentales que permitieron resolver el laboratorio y que reflejen su particularidad única. (Sólo en trabajos grupales se permite duplicidad)
  - Ejecución: Muestra comandos, capturas de pantalla, explicando la forma de replicar y ejecutar el entregable del laboratorio.

4.2. Pregunta

- ¿Cómo podríamos saber con exactitud, cuál de todas las propuestas algorítmicas es la más óptima?. Explíque.

4.3. Entregables

- URL al directorio específico del laboratorio en su repositorio GitHub privado donde esté todo el código fuente y otros que sean necesarios. Evitar la presencia de archivos: binarios, objetos, archivos temporales, cache, librerias, entornos virtuales. Si hay configuraciones particulares puede incluir archivos de especificación como: packages.json, requirements.txt o README.md.
- No olvide que el profesor debe ser siempre colaborador a su repositorio que debe ser privado (Usuario del profesor **@rescobedoulasalle**).
- El informe debe estar elabórado en LaTeX
- Usted debe describir sólo los **commits** más importantes que marcaron hitos en su trabajo, adjutando capturas de pantalla, del commit, porciones de código fuente, evidencia de sus ejecuciones y pruebas.
- En el informe siempre se debe explicar las imágenes (código fuente, capturas de pantalla, commits, ejecuciones, pruebas, etc.) con descripciones puntuales pero precisas.
- Agregar la estructura de directorios y archivos de su laboratorio hasta el nivel 4.

```
r.escobedo/
|--- lp1
|   |--- README.md
|   |--- .gitignore
|   |--- lab01
|       |--- README.md
|       |--- .gitignore
|       |--- DigitalRoot_01.cpp
|       |--- DigitalRoot_02.cpp
|       |--- DigitalRoot_03.cpp
|       |--- DigitalRoot_04.cpp
|       |--- DigitalRoot_05.cpp
|       |--- exercises
|           |--- DigitalRoot_01.java
|           |--- DigitalRoot_02.java
|           |--- DigitalRoot_03.java
|           |--- DigitalRoot_04.java
|           |--- DigitalRoot_05.java
|       |--- latex
|           |--- r.escobedo_lp1_26a_lab01.tex
|           |--- r.escobedo_lp1_26a_lab01.pdf
```

5. Rúbricas

5.1. Contenido del Informe

- El alumno deberá autocalificarse, marcando o dejando en blanco las celdas de la columna **Checklist**, deacuerdo a si cumplió o no con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación siempre será sobre la nota mínima aprobatoria, siempre y cuando cumpla con todos lo items. (Máximo 24 horas)
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la tabla de calificación de niveles de desempeño:

Tabla 3: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 4: Rúbrica para contenido del Informe y evidencias

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Repositorio se pudo clonar y se evidencia la estructura adecuada para revisar los entregables. (Se descontará puntos por error o observación)	4		1	
2. Commits	Hay porciones de código fuente asociado a los commits planificados con explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4		1	
3. Ejecución	Se incluyen comandos de ejecuciones y pruebas del ejecutable asociadas a código fuente explicadas gradualmente que permitirían replicar el proyecto. (Se descontará puntos por cada omisión)	4		1	
4. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2		0	
7. Ortografía	El documento no muestra errores ortográficos. (Se descontará puntos por error encontrado)	2		1	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente con explicaciones puntuales pero precisas, agregando diagramas generados a partir del código fuente y refleja un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4		1	
Total		20			

6. Referencias

- https://omegaup.com/arena/problem/Contratacion-Dinamica/#problems